PACKAGE BY LAYER

The first, and perhaps simplest, design approach is the traditional horizontal layered architecture, where we separate our code based on what it does from a technical perspective. This is often called "package by layer." Figure 34.1 shows what this might look like as a UML class diagram.

In this typical layered architecture, we have one layer for the web code, one layer for our "business logic," and one layer for persistence. In other words, code is sliced horizontally into layers, which are used as a way to group similar types of things. In a "strict layered architecture," layers should depend only on the next adjacent lower layer. In Java, layers are typically implemented as packages. As you can see in Figure 34.1, all of the dependencies between layers (packages) point downward. In this example, we have the following Java types:

- `OrdersController`: A web controller, something like a Spring MVC controller, that handles requests from the web.
- `OrdersService`: An interface that defines the "business logic" related to orders.
- `OrdersServiceImpl`: The implementation of the orders service.[1]
- `OrdersRepository`: An interface that defines how we get access to persistent order information.
- `JdbcOrdersRepository`: An implementation of the repository interface.

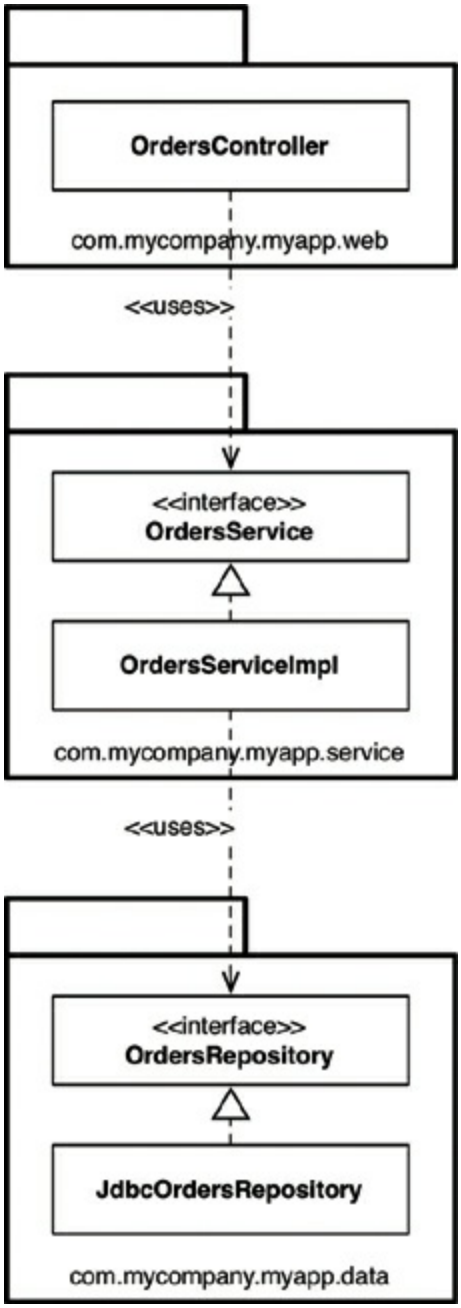

Figure 34.1 Package by layer

In "Presentation Domain Data Layering,"[2] Martin Fowler says that adopting such a layered architecture is a good way to get started. He's not alone. Many of the books, tutorials, training courses, and sample code you'll find will also point you down the path of creating a layered architecture. It's a very quick way to get something up and running without a huge amount of complexity. The problem, as Martin points out, is that once your software grows in scale and complexity, you will quickly find that having three large buckets of code isn't sufficient, and you will need to think about modularizing further.

Another problem is that, as Uncle Bob has already said, a layered architecture doesn't scream anything about the business domain. Put the code for two layered architectures, from two very different business domains, side by side and they will likely look eerily similar: web, services, and repositories. There's also another huge problem with layered architectures, but we'll get to that later.

PACKAGE BY FEATURE

Another option for organizing your code is to adopt a "package by feature" style. This is a vertical slicing, based on related features, domain concepts, or aggregate roots (to use domain-driven design terminology). In the typical implementations that I've seen, all of the types are placed into a single Java package, which is named to reflect the concept that is being grouped.

With this approach, as shown in Figure 34.2, we have the same interfaces and classes as before, but they are all placed into a single Java package rather than being split among three packages. This is a very simple refactoring from the "package by layer" style, but the top-level organization of the code now screams something about the business domain. We can now see that this code base has something to do with orders rather than the web, services, and repositories. Another benefit is that it's potentially easier to find all of the code that you need to modify in the event that the "view orders" use case changes. It's all sitting in a single Java package rather than being spread out.[3]

I often see software development teams realize that they have problems with horizontal layering ("package by layer") and switch to vertical layering ("package by feature") instead. In my opinion, both are suboptimal. If you've read this book so far, you might be thinking that we can do much better—and you're right.

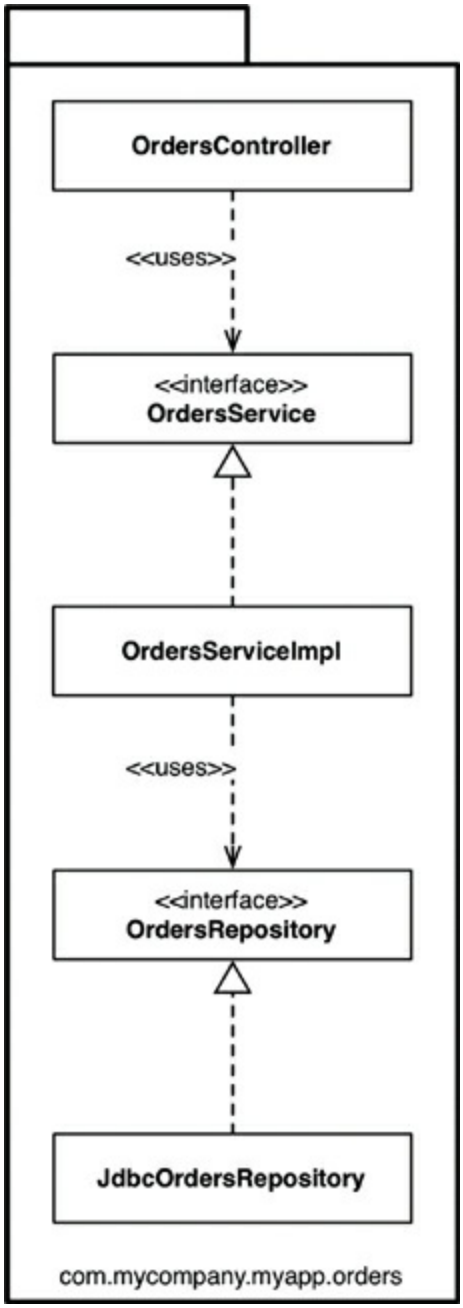

Figure 34.2 Package by feature

PORTS AND ADAPTERS

As Uncle Bob has said, approaches such as “ports and adapters,” the “hexagonal architecture,” “boundaries, controllers, entities,” and so on aim to create architectures where business/domain-focused code is independent and separate from the technical implementation details such as frameworks and databases. To summarize, you often see such code bases being composed of an “inside” (domain) and an “outside” (infrastructure), as suggested in Figure 34.3.

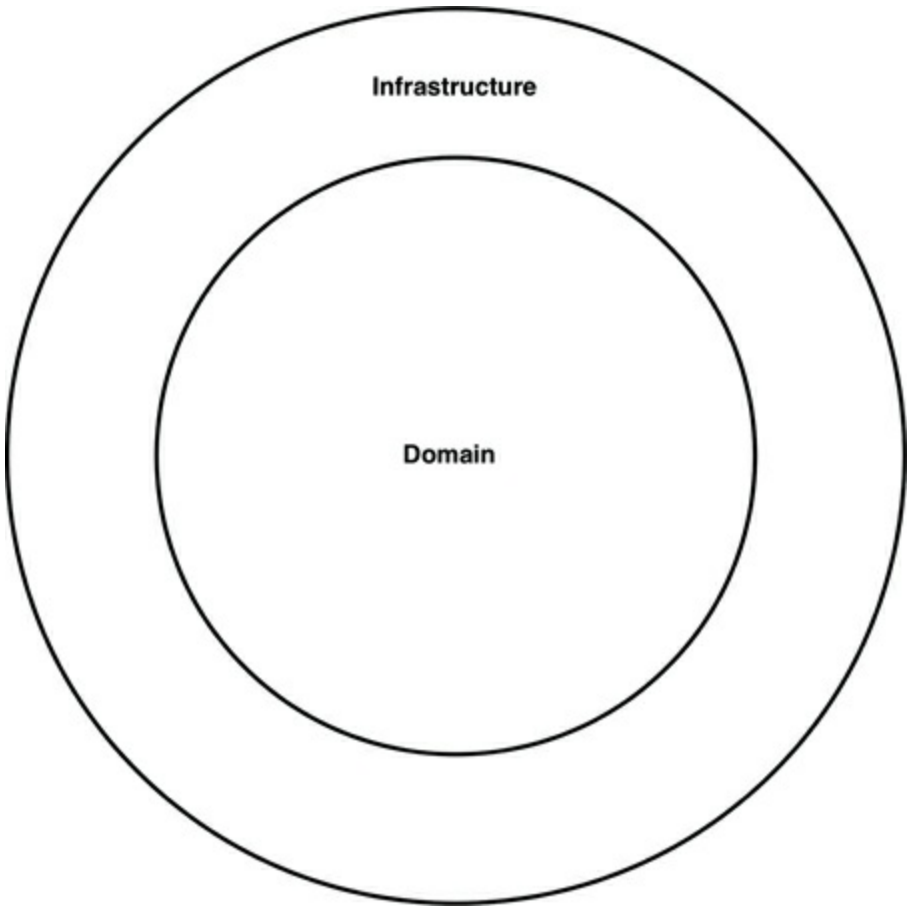

Figure 34.3 A code base with an inside and an outside

The “inside” region contains all of the domain concepts, whereas the “outside” region contains the interactions with the outside world (e.g., UIs, databases, third-party integrations). The major rule here is that the “outside” depends on the “inside”—never the other way around. Figure 34.4 shows a version of how the “view orders” use case might be implemented.

The `com.mycompany.myapp.domain` package here is the “inside,” and the other packages are the “outside.” Notice how the dependencies flow toward the “inside.” The keen-eyed reader will notice that the `OrdersRepository` from previous diagrams has been renamed to simply be `Orders`. This comes from the world of domain-driven design, where the advice is that the naming of everything on the “inside” should be stated in terms of the “ubiquitous domain language.” To put that another way, we talk about “orders” when we’re having a discussion about the domain, not the “orders repository.”

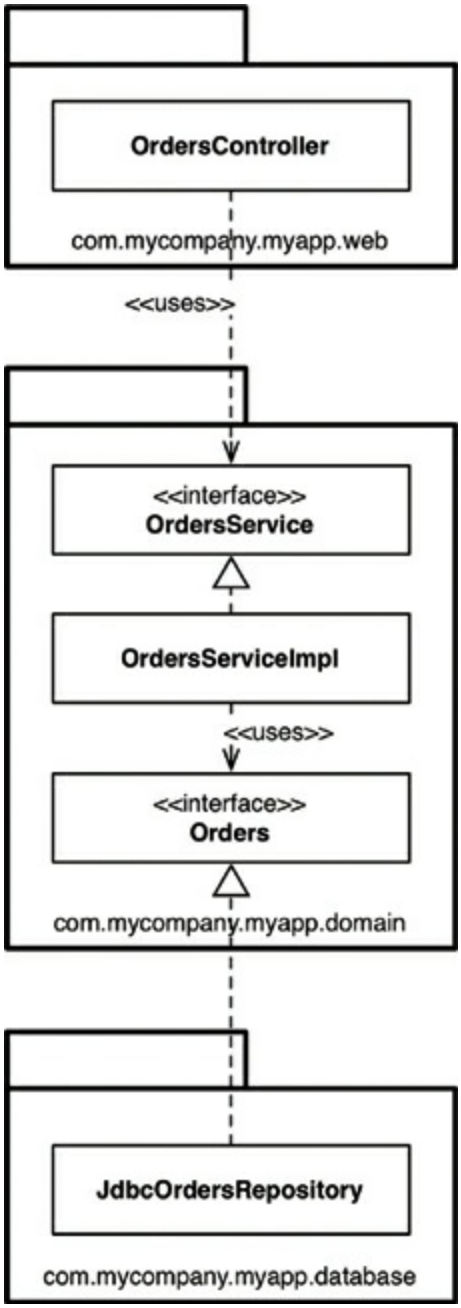

Figure 34.4 View orders use case

It's also worth pointing out that this is a simplified version of what the UML class diagram might look like, because it's missing things like interactors and objects to marshal the data across the dependency boundaries.

PACKAGE BY COMPONENT

Although I agree wholeheartedly with the discussions about SOLID, REP, CCP, and CRP and most of the advice in this book, I come to a slightly different conclusion

about how to organize code. So I'm going to present another option here, which I call "package by component." To give you some background, I've spent most of my career building enterprise software, primarily in Java, across a number of different business domains. Those software systems have varied immensely, too. A large number have been web-based, but others have been client–server[4], distributed, message-based, or something else. Although the technologies differed, the common theme was that the architecture for most of these software systems was based on a traditional layered architecture.

I've already mentioned a couple of reasons why layered architectures should be considered bad, but that's not the whole story. The purpose of a layered architecture is to separate code that has the same sort of function. Web stuff is separated from business logic, which is in turn separated from data access. As we saw from the UML class diagram, from an implementation perspective, a layer typically equates to a Java package. From a code accessibility perspective, for the `OrdersController` to be able to have a dependency on the `OrdersService` interface, the `OrdersService` interface needs to be marked as `public`, because they are in different packages. Likewise, the `OrdersRepository` interface needs to be marked as `public` so that it can be seen outside of the repository package, by the `OrdersServiceImpl` class.

In a strict layered architecture, the dependency arrows should always point downward, with layers depending only on the next adjacent lower layer. This comes back to creating a nice, clean, acyclic dependency graph, which is achieved by introducing some rules about how elements in a code base should depend on each other. The big problem here is that we can cheat by introducing some undesirable dependencies, yet still create a nice, acyclic dependency graph.

Suppose that you hire someone new who joins your team, and you give the newcomer another `orders`-related use case to implement. Since the person is new, he wants to make a big impression and get this use case implemented as quickly as possible. After sitting down with a cup of coffee for a few minutes, the newcomer discovers an existing `OrdersController` class, so he decides that's where the code for the new `orders`-related web page should go. But it needs some `orders` data from the database. The newcomer has an epiphany: "Oh, there's an `OrdersRepository` interface already built, too. I can simply dependency-inject the implementation into my controller. Perfect!" After a few more minutes of hacking, the web page is working. But the resulting UML diagram looks like Figure 34.5.

The dependency arrows still point downward, but the `OrdersController` is now additionally bypassing the `OrdersService` for some use cases. This organization is

often called a *relaxed layered architecture*, as layers are allowed to skip around their adjacent neighbor(s). In some situations, this is the intended outcome—if you're trying to follow the CQRS[5] pattern, for example. In many other cases, bypassing the business logic layer is undesirable, especially if that business logic is responsible for ensuring authorized access to individual records, for example.

While the new use case works, it's perhaps not implemented in the way that we were expecting. I see this happen a lot with teams that I visit as a consultant, and it's usually revealed when teams start to visualize what their code base really looks like, often for the first time.

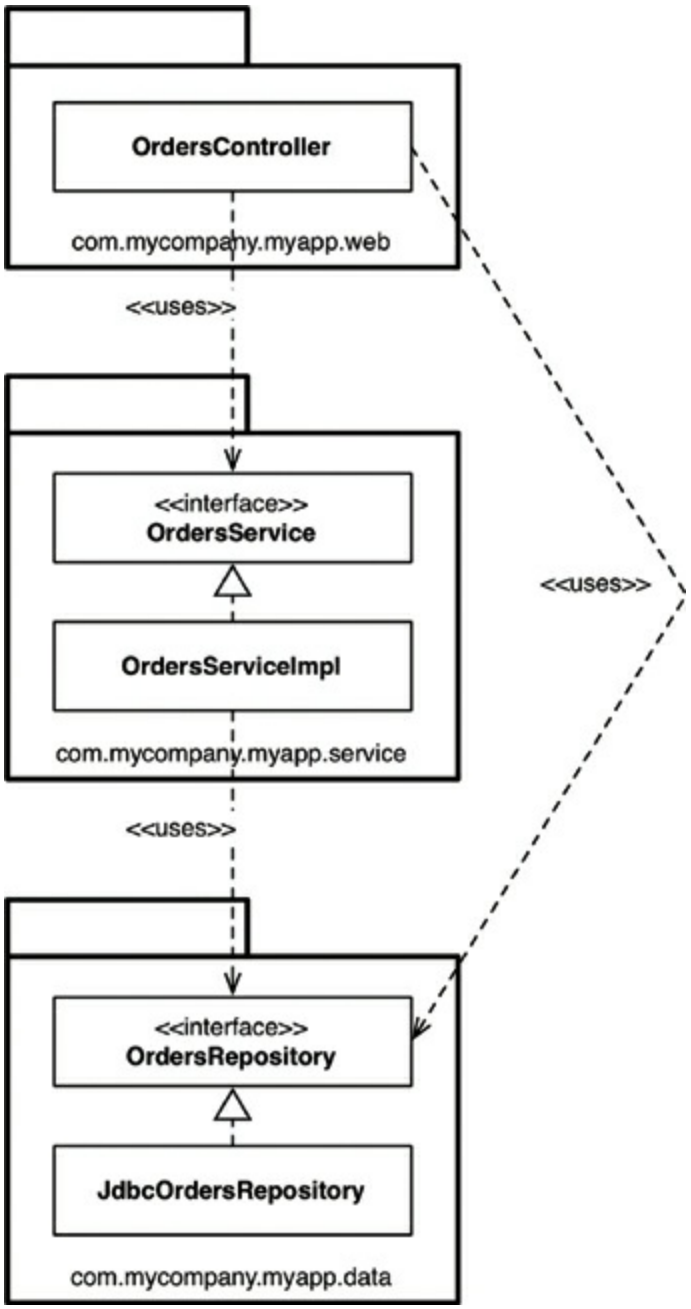

Figure 34.5 Relaxed layered architecture

What we need here is a guideline—an architectural principle—that says something like, "Web controllers should never access repositories directly." The question, of course, is enforcement. Many teams I've met simply say, "We enforce this principle through good discipline and code reviews, because we trust our developers." This confidence is great to hear, but we all know what happens when budgets and deadlines start looming ever closer.

A far smaller number of teams tell me that they use static analysis tools (e.g., NDepend, Structure101, Checkstyle) to check and automatically enforce architecture violations at build time. You may have seen such rules yourself; they usually manifest themselves as regular expressions or wildcard strings that state "types in package `**/web` should not access types in `**/data`"; and they are executed after the compilation step.

This approach is a little crude, but it can do the trick, reporting violations of the architecture principles that you've defined as a team and (you hope) failing the build. The problem with both approaches is that they are fallible, and the feedback loop is longer than it should be. If left unchecked, this practice can turn a code base into a "big ball of mud."[6] I'd personally like to use the compiler to enforce my architecture if at all possible.

This brings us to the "package by component" option. It's a hybrid approach to everything we've seen so far, with the goal of bundling all of the responsibilities related to a single coarse-grained component into a single Java package. It's about taking a service-centric view of a software system, which is something we're seeing with micro-service architectures as well. In the same way that ports and adapters treat the web as just another delivery mechanism, "package by component" keeps the user interface separate from these coarse-grained components. Figure 34.6 shows what the "view orders" use case might look like.

In essence, this approach bundles up the "business logic" and persistence code into a single thing, which I'm calling a "component." Uncle Bob presented his definition of "component" earlier in the book, saying:

> *Components are the units of deployment. They are the smallest entities that can be deployed as part of a system. In Java, they are jar files.*

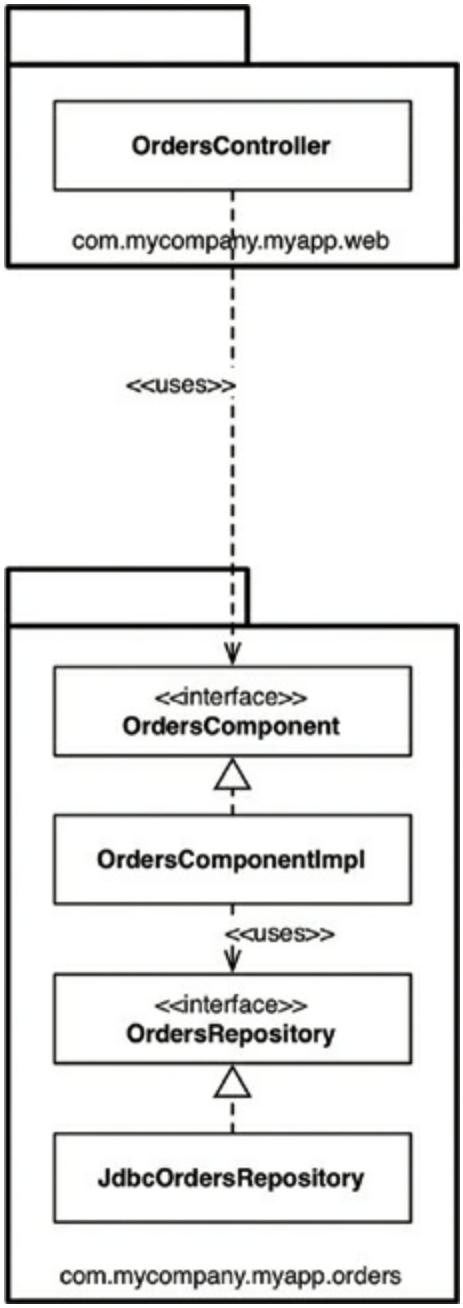

Figure 34.6 View orders use case

My definition of a component is slightly different: "A grouping of related functionality behind a nice clean interface, which resides inside an execution environment like an application." This definition comes from my "C4 software architecture model,"[7] which is a simple hierarchical way to think about the static structures of a software system in terms of containers, components, and classes (or code). It says that a software system is made up of one or more containers (e.g., web applications, mobile apps, stand-alone applications, databases, file systems), each of which contains one or more components, which in turn are implemented by one or more classes (or code). Whether each component resides in a separate jar file is an